Module 10: Agentic AI, LLMOps, Cloud Deployment and Privacy

1. Agentic AI Foundations

Checklist

☐ **Has a planner** — The system can decide what steps are required before producing the final response.

☐ **Has at least two tools** — The agent can perform useful external actions instead of relying only on generated text.

☐ **Memory** — The system can retain relevant conversation, user or workflow information across steps.

☐ **Retry** — Failed model or tool calls can be attempted again according to a controlled policy.

☐ **Reflection** — The agent can inspect its intermediate result and decide whether correction is required.

☐ **Human approval** — High-risk actions require confirmation before execution.

☐ **Structured output** — The final response follows a predictable schema such as JSON or Pydantic.

☐ **Error handling** — Tool failures, invalid inputs and model errors are handled without crashing the system.

☐ **Logging** — Important requests, actions, failures and outputs are recorded for debugging and monitoring.

Metrics

Tool Selection Accuracy

Measures how often the agent chooses the correct tool.

$$\text{Tool Selection Accuracy} = \frac{\text{Correct Tool Selections}}{\text{Total Tool Decisions}}$$

Task Success Rate

Measures how often the agent successfully completes the user's actual objective.

$$\text{Task Success Rate} = \frac{\text{Successful Tasks}}{\text{Total Tasks}}$$

Step Efficiency

Measures how close the agent's execution is to the minimum required number of steps.

$$\text{Step Efficiency} = \frac{\text{Minimum Required Steps}}{\text{Actual Steps}}$$

A value closer to 1 is better.

Tool Success Rate

Measures how often external tools execute successfully.

$$\text{Tool Success Rate} = \frac{\text{Successful Tool Calls}}{\text{Total Tool Calls}}$$

Loop Rate

Measures how often the agent gets stuck repeating the same action or reasoning pattern.

$$\text{Loop Rate} = \frac{\text{Executions Entering a Loop}}{\text{Total Executions}}$$

---

2. LangChain, LangGraph and CrewAI

Concepts

LangChain

Components — Reusable building blocks such as models, tools, prompts, retrievers and parsers.

Chains — Fixed sequences where the output of one component becomes the input of another.

Agents — LLM-driven systems that dynamically decide which actions to take.

Tools — Functions or services the agent can invoke.

Memory — Mechanisms for retaining context between interactions.

LangGraph

Nodes — Individual processing steps such as planner, tool, validator or human review.

Edges — Connections that determine how execution moves between nodes.

State — Shared data maintained throughout the workflow.

Workflow — The complete sequence of connected nodes and decisions.

Conditional routing — Execution moves to different nodes based on runtime conditions.

CrewAI

Role — The defined responsibility of an individual agent.

Goal — The outcome that agent is expected to achieve.

Backstory — Context used to shape the agent's behaviour and expertise.

Task — A specific unit of work assigned to an agent.

Agent collaboration — Multiple agents share work or pass results to one another.

Checklist

☐ **Tool abstraction** — Tools have standard names, descriptions, inputs and outputs.

☐ **Prompt templates** — Prompts are reusable, parameterised and version-controlled.

☐ **State management** — Workflow data is preserved consistently across steps.

☐ **Retry** — Failed nodes or tool calls can be safely repeated.

☐ **Conditional routing** — The workflow can take different paths based on context or results.

☐ **Human node** — A person can review, approve or correct the workflow.

☐ **Parallel execution** — Independent tasks can run simultaneously to reduce latency.

☐ **Multi-agent design** — Multiple agents are used only when they have genuinely distinct responsibilities.

Metrics

Workflow Completion Rate — Percentage of workflows that reach the intended final state.

$$\text{Workflow Completion Rate} = \frac{\text{Completed Workflows}}{\text{Total Workflows}}$$

Agent Handoff Accuracy — Percentage of agent-to-agent transfers that include the correct task and context.

$$\text{Agent Handoff Accuracy} = \frac{\text{Correct Handoffs}}{\text{Total Handoffs}}$$

Node Success Rate — Percentage of workflow nodes that execute successfully.

$$\text{Node Success Rate} = \frac{\text{Successful Node Executions}}{\text{Total Node Executions}}$$

Average Node Latency — Average execution time of a workflow node.

$$\text{Average Node Latency} = \frac{\sum \text{Node Execution Time}}{\text{Total Node Executions}}$$

Agent Idle Time — Time an agent spends waiting for another agent, tool or human response.

---

3. Practical Agent Integration

Concepts

API — A service interface used by the agent to retrieve data or perform an action.

Calculator — A deterministic tool used for exact numerical computation.

Weather — An external real-time data tool used for current forecasts or conditions.

SQL — A structured database query tool for retrieving or updating records.

RAG — Retrieval-Augmented Generation combines external knowledge retrieval with LLM generation.

Email — A tool for reading, drafting or sending messages under controlled permissions.

GitHub — A tool for accessing repositories, issues, commits or pull requests.

PDF — A document source that may be parsed, searched or summarised.

OCR — Optical Character Recognition converts text inside images or scanned documents into machine-readable text.

Vision — Image understanding used for classification, detection or multimodal analysis.

Speech — Audio input or output through speech recognition and speech synthesis.

Checklist

☐ **Every tool documented** — Each tool explains what it does and when it should be used.

☐ **Input schema** — Tool arguments have defined names, types and validation rules.

☐ **Output schema** — Tool results follow a predictable structure.

☐ **Retry** — Temporary failures can be retried safely.

☐ **Timeout** — Tool calls stop after a defined maximum duration.

☐ **Authentication** — Only authorised systems or users can call the tool.

☐ **Cost** — Each tool's monetary or computational cost is tracked.

☐ **Latency** — Execution time is measured for each tool.

☐ **Security** — Tools enforce permissions and validate inputs.

Metrics

API Success Rate

$$\text{API Success Rate} = \frac{\text{Successful API Calls}}{\text{Total API Calls}}$$

Retry Success Rate

$$\text{Retry Success Rate} = \frac{\text{Successful Recoveries After Retry}}{\text{Total Retried Calls}}$$

Timeout Rate

$$\text{Timeout Rate} = \frac{\text{Timed-Out Calls}}{\text{Total Calls}}$$

Argument Accuracy

$$\text{Argument Accuracy} = \frac{\text{Tool Calls With Correct Arguments}}{\text{Total Tool Calls}}$$

---

4. Retrieval-Augmented Generation

Concepts

Chunking — Documents are divided into smaller searchable units.

Embedding — Text or images are converted into numerical vectors representing semantic meaning.

Similarity search — The system finds stored vectors most similar to the user query.

Vector database — A database optimised for storing and searching embeddings.

Metadata — Additional information such as title, date, author, tenant or category attached to each chunk.

Hybrid search — Combines semantic vector search with keyword or lexical search.

Re-ranking — A secondary model reorders retrieved results by relevance.

Grounding — The generated answer is supported by retrieved evidence.

Architecture

Question
↓
Embedding
↓
Vector search
↓
Top-K results
↓
Prompt with evidence
↓
LLM answer

Checklist

☐ **Chunking** — Chunk size and overlap are appropriate for the document type.

☐ **Metadata** — Retrieved chunks retain source and filtering information.

☐ **Embedding** — The selected embedding model matches the language and domain.

☐ **Vector database** — Embeddings are stored and searched efficiently.

☐ **Citation** — Answers link factual claims to supporting evidence.

☐ **Source display** — Users can inspect the original supporting source.

☐ **Hybrid search** — Keyword and semantic search are combined when useful.

☐ **Re-ranking** — Retrieved results are reordered using a stronger relevance model.

Metrics

Precision@K

Measures the fraction of top-K retrieved items that are relevant.

$$\text{Precision@K} = \frac{\text{Relevant Items in Top K}}{K}$$

Recall@K

Measures how many of all relevant items were retrieved in the top K.

$$\text{Recall@K} = \frac{\text{Relevant Items Retrieved in Top K}}{\text{Total Relevant Items}}$$

Hit Rate@K

Measures whether at least one relevant result appears in the top K.

$$\text{Hit Rate@K} = \frac{\text{Queries With At Least One Relevant Result}}{\text{Total Queries}}$$

Mean Reciprocal Rank

Rewards systems where the first relevant result appears near the top.

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{Rank of First Relevant Result}_i}$$

Groundedness

Measures the percentage of factual claims supported by retrieved evidence.

$$\text{Groundedness} = \frac{\text{Supported Factual Claims}}{\text{Total Factual Claims}}$$

Citation Accuracy

Measures whether citations genuinely support the attached claims.

$$\text{Citation Accuracy} = \frac{\text{Correct Supporting Citations}}{\text{Total Citations}}$$

---

5. Structured Outputs

Concepts

JSON — A machine-readable key-value format commonly used by APIs.

Pydantic — A Python validation library used to define and enforce output schemas.

Validation — Checking whether the output satisfies type, format and business constraints.

Schema — A formal definition of required fields and data types.

Parsing — Converting raw output into a structured object.

Serialization — Converting an object into JSON or another transferable format.

Checklist

☐ **JSON output** — Responses can be consumed by other software.

☐ **Validation** — Invalid values are rejected or corrected.

☐ **Pydantic model** — Python schemas define expected fields and types.

☐ **Required fields** — Mandatory data cannot be omitted.

☐ **Error messages** — Validation failures produce understandable feedback.

Metrics

Schema Compliance Rate

Measures how many outputs pass schema validation.

$$\text{Schema Compliance Rate} = \frac{\text{Valid Structured Responses}}{\text{Total Responses}}$$

Field Accuracy

Measures how many required field values are correct.

$$\text{Field Accuracy} = \frac{\text{Correct Field Values}}{\text{Total Required Field Values}}$$

---

6. Classification Evaluation

Concepts

True Positive — TP — The system predicted positive and reality was positive.

False Positive — FP — The system predicted positive but reality was negative.

True Negative — TN — The system predicted negative and reality was negative.

False Negative — FN — The system predicted negative but reality was positive.

Metrics

Accuracy

Measures the percentage of all predictions that are correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Measures how often positive predictions are correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Measures how many actual positive cases were found.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Specificity

Measures how many actual negative cases were correctly rejected.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

F1 Score

Balances precision and recall using the harmonic mean.

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Checklist

☐ **Confusion matrix** — Shows TP, FP, TN and FN together.

☐ **Accuracy** — Use when class distribution and failure costs are reasonably balanced.

☐ **Precision** — Use when false positives are expensive.

☐ **Recall** — Use when false negatives are dangerous.

☐ **F1** — Use when both precision and recall matter.

☐ **Macro average** — Gives equal importance to every class.

$$\text{Macro F1} = \frac{1}{K} \sum_{i=1}^{K} F1_i$$

☐ **Weighted average** — Weights each class by its number of examples.

$$\text{Weighted F1} = \frac{\sum_{i=1}^{K} n_i F1_i}{\sum_{i=1}^{K} n_i}$$

---

7. Agent Evaluation

Checklist

☐ **Tool selection** — Verify whether the agent chooses the correct tool.

☐ **Tool arguments** — Verify whether correct values are passed to the selected tool.

☐ **Planning** — Check whether the agent creates an effective sequence of actions.

☐ **Memory** — Check whether relevant context is remembered without storing unnecessary data.

☐ **Hallucination** — Detect unsupported or invented claims.

☐ **Grounding** — Verify whether claims follow from tool results or retrieved evidence.

☐ **Task success** — Determine whether the user's objective was actually completed.

☐ **Human approval** — Confirm that protected actions cannot bypass human review.

Metrics

Task Success Rate — Percentage of tasks completed correctly.

Tool Selection Accuracy — Percentage of tool decisions that choose the expected tool.

Average Steps

$$\text{Average Steps} = \frac{\text{Total Executed Steps}}{\text{Total Agent Runs}}$$

Loop Count — Number of repeated reasoning or action cycles.

Completion Time — Total time from user request to final result.

---

8. Human Evaluation

Checklist

☐ **Correctness** — Is the answer factually and logically right?

☐ **Helpfulness** — Does the answer help the user complete the intended task?

☐ **Completeness** — Does it cover every required part?

☐ **Safety** — Does it avoid harmful, unauthorised or risky behaviour?

☐ **Tone** — Is the communication appropriate for the user and context?

☐ **Groundedness** — Are factual claims supported by evidence?

☐ **Citation quality** — Do citations point to the correct supporting sources?

Rubric

1–5 rating scale — Reviewers assign a score from poor to excellent using defined criteria.

Likert scale — Reviewers indicate degrees of agreement, such as strongly disagree to strongly agree.

Agreement

Inter-Annotator Agreement — Measures how consistently multiple human reviewers judge the same outputs.

Basic agreement:

$$\text{Agreement Rate} = \frac{\text{Matching Reviewer Judgments}}{\text{Total Judgments}}$$

---

9. Debugging

Pipeline

Input
↓
Planner
↓
Retriever
↓
Tool
↓
LLM
↓
Output

Checklist

☐ **Trace** — A complete record of every step in one request.

☐ **Prompt** — The exact prompt and version used must be recorded.

☐ **Tool logs** — Tool names, arguments, outputs and failures must be visible.

☐ **Token logs** — Input and output token counts must be tracked.

☐ **Error logs** — Errors must include type, time and request context.

☐ **Stack trace** — Technical exceptions must show where the code failed.

☐ **Root cause** — The underlying component responsible for the failure must be identified.

Error Taxonomy

Input error — User input is missing, invalid, ambiguous or malicious.

Intent error — The system misunderstands what the user wants.

Planner error — The system chooses an incorrect sequence of actions.

Tool error — A tool is incorrectly selected or fails during execution.

Retriever error — Relevant evidence is not found.

Memory error — Relevant context is forgotten or irrelevant context is used.

Prompt error — Instructions are unclear, conflicting or incomplete.

Reasoning error — The model misinterprets valid information.

Output error — The result is incomplete, invalid or wrongly formatted.

Deployment error — Infrastructure, networking or configuration prevents correct operation.

---

10. Observability

Concepts

Tracing — Following one request across models, tools, databases and services.

Logging — Recording detailed events and errors.

Metrics — Numerical measurements aggregated over many requests.

Alerts — Notifications triggered when metrics cross defined thresholds.

Dashboards — Visual summaries of system health and performance.

Checklist

☐ **Prompt logs** — Record the prompt and prompt version.

☐ **Tool logs** — Record selected tools, arguments and results.

☐ **Token usage** — Track input, output and total tokens.

☐ **Latency** — Measure response and component execution times.

☐ **Errors** — Track failures by type and frequency.

☐ **Cost** — Track model, tool and infrastructure cost.

☐ **User feedback** — Record ratings, corrections and rejection signals.

Metrics

P50 latency — Half of all requests finish faster than this value.

P95 latency — Ninety-five percent of requests finish faster than this value.

P99 latency — Ninety-nine percent of requests finish faster than this value.

Error Rate

$$\text{Error Rate} = \frac{\text{Failed Requests}}{\text{Total Requests}}$$

Availability

$$\text{Availability} = \frac{\text{Uptime}}{\text{Total Scheduled Time}}$$

---

11. LLMOps

Concepts

Versioning — Tracking changes to prompts, models, datasets and configurations.

Experiments — Comparing alternative models, prompts or workflows.

Evaluation — Measuring quality against a defined test dataset.

CI/CD — Automatically testing and deploying approved changes.

Rollback — Returning to a previous stable version after failure.

Deployment — Releasing the system into a usable environment.

Feedback loop — Using real failures and user feedback to improve the evaluation dataset.

Monitoring — Continuously observing production quality, cost and reliability.

Checklist

☐ **Prompt version** — Every prompt change has a version identifier.

☐ **Dataset version** — Evaluation datasets are reproducible.

☐ **Model version** — Model provider and model revision are recorded.

☐ **Evaluation pipeline** — Every meaningful change runs automated tests.

☐ **A/B testing** — Two versions can be compared using real or controlled traffic.

☐ **Rollback** — A failed release can be reversed quickly.

☐ **Monitoring** — Production quality, latency, cost and failures are continuously tracked.

Metrics

Regression Rate — Percentage of previously working tests that fail after a change.

$$\text{Regression Rate} = \frac{\text{Previously Passing Cases Now Failing}}{\text{Previously Passing Cases}}$$

Acceptance Rate

$$\text{Acceptance Rate} = \frac{\text{Responses Accepted Without Correction}}{\text{Total Responses}}$$

Failure Rate

$$\text{Failure Rate} = \frac{\text{Failed Tasks}}{\text{Total Tasks}}$$

Deployment Frequency — Number of successful production releases during a defined period.

---

12. Cloud Deployment

Concepts

FastAPI — A Python framework for exposing the AI system through HTTP APIs.

Docker — Packages application code and dependencies into a portable container.

AWS — Amazon’s cloud platform.

EC2 — Virtual servers offering full control over compute resources.

Lambda — Serverless compute for short, event-driven tasks.

Bedrock — AWS-managed access to foundation models.

SageMaker — AWS platform for training, deploying and monitoring machine-learning models.

Vertex AI — Google Cloud’s managed AI development and deployment platform.

Azure AI — Microsoft’s managed AI and enterprise integration platform.

GPU — Hardware designed for parallel model training and inference.

Autoscaling — Automatically adding or removing resources based on workload.

Checklist

☐ **Docker** — The application can run consistently across environments.

☐ **API** — Core functionality is accessible through a stable service interface.

☐ **HTTPS** — Data is encrypted during network transmission.

☐ **Secrets** — API keys and passwords are stored outside source code.

☐ **Load balancer** — Traffic is distributed across multiple service instances.

☐ **Autoscaling** — Capacity adjusts to traffic.

☐ **Monitoring** — Infrastructure and application health are measured.

☐ **Logging** — Deployment and runtime events are recorded centrally.

Metrics

Requests per second — Number of requests handled each second.

Latency — Time required to complete a request.

Availability — Percentage of time the service is operational.

Cost per hour — Infrastructure cost for each operating hour.

CPU utilisation — Percentage of available processor capacity being used.

GPU utilisation — Percentage of GPU processing capacity being used.

Memory utilisation — Percentage of available RAM being used.

---

13. Privacy, Security and Responsible AI

Concepts

PII — Personally identifiable information such as names, phone numbers, addresses or identity numbers.

GDPR — European data-protection regulation governing personal-data processing.

DPDP — India's Digital Personal Data Protection framework.

HIPAA — US regulation protecting certain healthcare information.

RBAC — Role-Based Access Control assigns permissions based on user roles.

Encryption — Converts data into a protected form unreadable without the correct key.

Consent — Users understand and approve how their data will be used.

Secrets — Sensitive credentials such as API keys, passwords and tokens.

Prompt injection — Malicious instructions attempt to manipulate the model through user or retrieved content.

Jailbreak — An attempt to bypass model safety rules.

Checklist

☐ **Authentication** — Confirm who the user is.

☐ **Authorization** — Confirm what that user is allowed to do.

☐ **PII detection** — Identify sensitive personal information in inputs and outputs.

☐ **Encryption** — Protect data in transit and at rest.

☐ **Secret management** — Store credentials in a secure secret manager.

☐ **RBAC** — Restrict actions based on user roles.

☐ **Human approval** — Require review before high-risk actions.

☐ **Audit logs** — Maintain a history of important security and data-access events.

Metrics

PII Recall

Measures how many real PII instances were detected.

$$\text{PII Recall} = \frac{\text{Detected PII Instances}}{\text{Total Actual PII Instances}}$$

Unauthorized Access Rate

$$\text{Unauthorized Access Rate} = \frac{\text{Successful Unauthorized Actions}}{\text{Unauthorized Attempts}}$$

The desired value is zero.

Prompt Injection Success Rate

$$\text{Prompt Injection Success Rate} = \frac{\text{Successful Injection Attacks}}{\text{Total Injection Attempts}}$$

False Refusal Rate

Measures how often valid requests are incorrectly blocked.

$$\text{False Refusal Rate} = \frac{\text{Safe Requests Incorrectly Refused}}{\text{Total Safe Requests}}$$

Data Leak Rate

$$\text{Data Leak Rate} = \frac{\text{Responses Exposing Protected Data}}{\text{Total Responses}}$$

The desired value is zero.

---

14. Production Readiness

Architecture

☐ **Diagram** — Show all major system components and connections.

☐ **Components** — Define the responsibility of every service or module.

☐ **Workflow** — Show the complete request and decision flow.

AI

☐ **Agent** — Define the system's overall autonomous goal.

☐ **Planner** — Define how the system selects steps.

☐ **Tools** — Document every external capability.

☐ **Memory** — Define what is stored and for how long.

☐ **RAG** — Explain why external retrieval is necessary.

Evaluation

☐ **Dataset** — Maintain representative normal, edge, failure and adversarial cases.

☐ **Metrics** — Select metrics based on the cost of failure.

☐ **Human evaluation** — Use explicit rubrics for subjective quality.

Debugging

☐ **Logs** — Record component events and failures.

☐ **Traces** — Follow complete request execution.

☐ **Errors** — Classify errors using a repeatable taxonomy.

Deployment

☐ **Docker** — Package the application reproducibly.

☐ **Cloud** — Select infrastructure based on workload requirements.

☐ **Monitoring** — Observe availability, quality, latency and cost.

Security

☐ **Authentication** — Verify user identity.

☐ **Authorization** — Enforce permissions outside the LLM.

☐ **Secrets** — Protect credentials.

☐ **Encryption** — Protect stored and transmitted data.

Reliability

☐ **Retry** — Retry transient failures safely.

☐ **Timeout** — Prevent requests from waiting indefinitely.

☐ **Fallback** — Use an alternative model, tool or response path.

☐ **Cache** — Reuse safe previous results to reduce latency and cost.

Cost

☐ **Tokens** — Track input and output usage.

☐ **Latency** — Measure end-to-end and component timing.

☐ **Model routing** — Use different models based on complexity, cost and risk.

☐ **Cache** — Avoid repeated expensive computation.

Documentation

☐ **README** — Explain the project, setup and usage.

☐ **API documentation** — Describe endpoints, inputs, outputs and errors.

☐ **Architecture documentation** — Explain components and technical decisions.

☐ **Demo** — Show successful, failing and recovery scenarios.

☐ **Future work** — List known limitations and realistic next improvements.

Production AI Design Review

Every team should answer these ten questions.

1. Why does this need an LLM?

Explain what requires flexible language understanding or reasoning and what could be solved with deterministic code.

2. What decisions are delegated to the LLM?

Clearly separate model decisions from permissions, calculations and business rules that must remain deterministic.

3. What are the five most likely failure modes?

Identify realistic failures in input, retrieval, tools, reasoning, output, security and infrastructure.

4. How will each failure be detected?

Associate every major failure with a metric, alert, validation rule, log or trace.

5. How will the system recover?

Define retry, timeout, fallback, cache, graceful failure and human-escalation behaviour.

6. How do you know the new version is better?

Compare versions using a fixed evaluation dataset and quality, latency and cost metrics.

7. How will user data and secrets be protected?

Explain authentication, authorization, encryption, retention, PII handling and secret storage.

8. What is the cost per successful task?

Calculate model, tool and infrastructure cost relative to successfully completed user outcomes.

$$\text{Cost per Successful Task} = \frac{\text{Total System Cost}}{\text{Successful Tasks}}$$

9. What breaks when users grow from 10 to 1 million?

Identify scaling limits in APIs, databases, model quotas, vector search, state, queues, cost and monitoring.

10. Would you trust the system as a customer?

Evaluate whether failures are visible, recoverable, safe and accountable enough for real-world use.